

Méthodologie

Yannis TALLY – Présentation projet 3



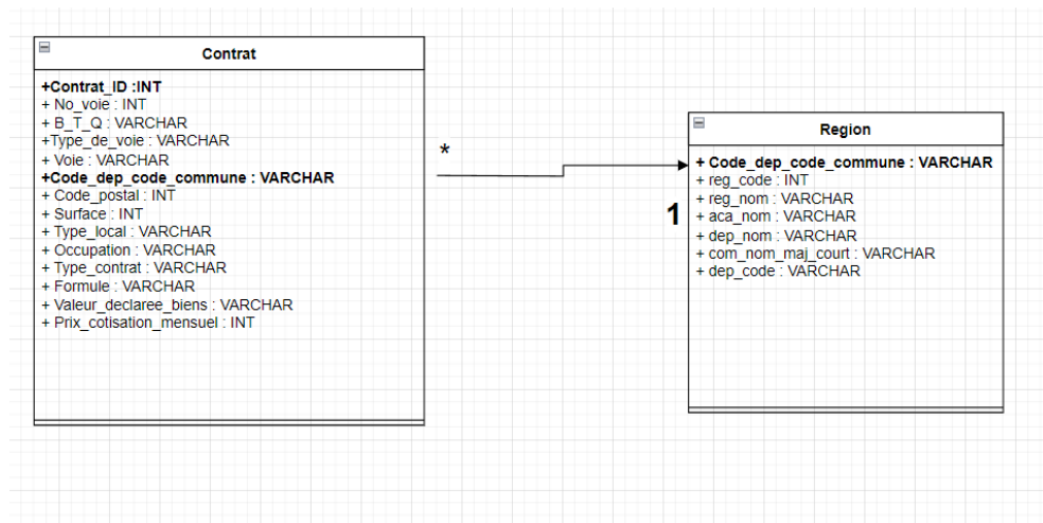
Dictionnaire de données

	Nom des colonnes	Type de données	Taille	Clé	Description
CONTRAT.CSV	Contrat_ID	INT	6	Clé primaire	Id unique pour les contrats
	No_voie	INT	5		Numéro dans la voie pour l'adresse du logement assuré
	B_T_Q	CHAR	1		Indicateur éventuel de répétition pour l'adresse du logement assuré sur un caractère
	Type_de_voie	CVARCHAR	4		Type de voie pour l'adresse du logement assuré: rue, av (Avenue), rte (Route), ...
	Voie	VARCHAR	50		Libellé de la voie pour l'adresse du logement assuré
	Code_dep_code_commune	VARCHAR	6	Clé secondaire	Concaténation du code département et code commune pour avoir une clé unique
	Code_postal	INT	5		Code postal pour l'adresse du logement assuré
	Surface	INT	3		Taille surface
	Type_local	VARCHAR	25		Type de local
	Occupation	VARCHAR	25		Type d'occupation
	Type_contrat	VARCHAR	25		Type de contrat
REGION.CSV	Formule	VARCHAR	25		Type de formule
	Valeur_declaree_biens	VARCHAR	25		Fourchette de prix
	Prix_cotisation_mensuel	INT	4		Cotisation mensuel
	Code_dep_code_commune	VARCHAR	6	Clé primaire	Concaténation du code département et code commune pour avoir une clé unique
	reg_code	INT	2		Code région
	reg_nom	VARCHAR	50		Nom région
	aca_nom	VARCHAR	50		Nom ville
	dep_nom	VARCHAR	50		Nom département
	com_nom_maj_court	VARCHAR	50		Nom commune
	dep_code	VARCHAR	25		Code département

Le dictionnaire de données est un atout précieux pour gérer efficacement ses données.

Schéma relationnel

Schéma relationnel :



Le schéma relationnel est un outil fondamental pour assurer une conception efficace et une gestion optimale de la base de données.

Base de données

Base de données :

```
CREATE DATABASE "projet_trois";
```

Table contrat :

```
CREATE TABLE `contrat` ( `Contrat_ID` int(11) NOT NULL AUTO_INCREMENT, `No_voie` int(10) NOT NULL, `B.T.Q` varchar(1) NOT NULL, `Type_de_voie` varchar(4) NOT NULL, `Voie` varchar(50) NOT NULL, `Code_dep_code_commune` varchar(6) NOT NULL, `Code_postal` int(7) NOT NULL, `Surface` int(7) NOT NULL, `Type_local` varchar(50) NOT NULL, `Occupation` varchar(50) NOT NULL, `Type_contrat` varchar(50) NOT NULL, `Formule` varchar(50) NOT NULL, `Valeur_declaree_biens` varchar(50) NOT NULL, `Prix_cotisation_mensuel` int(7) NOT NULL, PRIMARY KEY (`Contrat_ID`), KEY `Code_dep_code_commune` (`Code_dep_code_commune`), CONSTRAINT `contrat_ibfk_1` FOREIGN KEY (`Code_dep_code_commune`) REFERENCES `region` (`Code_dep_code_commune`) ON DELETE CASCADE ON UPDATE CASCADE )
```

Table region :

```
CREATE TABLE `region` ( `Code_dep_code_commune` varchar(6) NOT NULL, `reg_code` int(2) NOT NULL, `reg_nom` varchar(50) NOT NULL, `aca_nom` varchar(50) NOT NULL, `dep_nom` varchar(50) NOT NULL, `com_nom_maj_court` varchar(50) NOT NULL, `dep_code` varchar(50) NOT NULL, PRIMARY KEY (`Code_dep_code_commune`)
```

La base de données `projet\_trois` contient deux tables principales : `contrat` et `region`.

La table `contrat` stocke des informations détaillées sur des contrats, incluant des attributs tels que l'identifiant du contrat, le type de voie, la surface, et le prix de cotisation mensuel, tout en établissant une relation avec la table `region` via une clé étrangère.

La table `region` fournit des données géographiques, comme le code de la commune et le nom de la région, ce qui permet de lier les contrats à des localisations spécifiques.

Requêtes SQL et résultats

Requête 1: Lister les numéros de contrats (contrat_ID) avec leur surface pour la commune de Caen.

```
SELECT Contrat_ID, Surface FROM contrat
JOIN region
ON contrat.Code_dep_code_commune =
region.Code_dep_code_commune
WHERE com_nom_maj_court = "Caen";
```

Contrat_ID	Surface
103791	35
103792	99
103793	40
103794	20

Requête 1 et 2 jointures simples

Elles permettent de combiner des données provenant de plusieurs tables, facilitant ainsi l'accès à des informations connexes.

Requêtes SQL et résultats

Requête 4 : Quels sont les 5 contrats qui ont les surfaces les plus élevées ?

```
SELECT Contrat_ID, Surface FROM contrat
```

```
ORDER BY Surface DESC LIMIT 5;
```

▼	Contrat_ID	Surface	▼ 1
ner	104211	815	
ner	105463	742	
ner	130878	595	
ner	100822	570	
ner	109872	559	

Requête 3 et 4 ORDER BY – DESC - LIMIT

ORDER BY : Permet de trier les résultats d'une requête selon une ou plusieurs colonnes spécifiées.

DESC : Indique que le tri doit être effectué en ordre décroissant, c'est-à-dire du plus grand au plus petit.

LIMIT : Restreint le nombre de résultats retournés par une requête à un nombre spécifié.

Requêtes SQL et résultats

Requête 7 : Quel est le nombre de formules "integral" sur la région Pays de la Loire ?

```
SELECT COUNT(Formule) AS nbr_formule_integral, reg_nom FROM `contrat`  
JOIN region  
ON contrat.Code_dep_code_commune = region.Code_dep_code_commune  
WHERE Formule LIKE "Integral" AND reg_nom = "pays de la loire";
```

nbr_formule_integral	reg_nom
589	Pays de la Loire

Requête 4 à 12 calculs et utilisations des précédentes formules

- AVG : Calcule la moyenne des valeurs d'une colonne spécifiée dans une requête.
- HAVING : Filtre les résultats d'un groupe après l'application d'une fonction d'agrégation, comme COUNT ou AVG.
- GROUP BY : Regroupe les résultats d'une requête selon une ou plusieurs colonnes, permettant d'appliquer des fonctions d'agrégation.
- COUNT : Compte le nombre d'enregistrements ou d'éléments dans une colonne spécifiée.
- WHERE : Filtre les enregistrements d'une requête selon des conditions spécifiques avant l'agrégation.
- LIKE : Permet de rechercher des motifs dans une colonne de type texte, souvent utilisé avec des caractères génériques.
- AND : Combine plusieurs conditions dans une clause WHERE, nécessitant que toutes les conditions soient vraies pour inclure un enregistrement dans les résultats.

Merci



Yannis TALLY – Projet 3